3.1 Numbers

3.1.1 Types of Numbers

Four types of numbers are built into *Mathematica*.

`Integer`	arbitrary-length exact integer
`Rational`	*integer*`/`*integer* in lowest terms
`Real`	approximate real number, with any specified precision
`Complex`	complex number of the form *number* `+` *number* `I`

Intrinsic types of numbers in *Mathematica*.

Rational numbers always consist of a ratio of two integers, reduced to lowest terms.

```
In[1]:= 12344/2222
```

$$\text{Out[1]=}\ \frac{6172}{1111}$$

Approximate real numbers are distinguished by the presence of an explicit decimal point.

```
In[2]:= 5456.
Out[2]= 5456.
```

An approximate real number can have any number of digits.

```
In[3]:= 4.5454352345454352345345234523454 3
Out[3]= 4.5454352345454352345345234523454 3
```

Complex numbers can have integer or rational components.

```
In[4]:= 4 + 7/8 I
```

$$\text{Out[4]=}\ 4 + \frac{7\ \text{I}}{8}$$

They can also have approximate real number components.

```
In[5]:= 4 + 5.6 I
Out[5]= 4. + 5.6 I
```

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; http://www.wolfram.com/; 1-800-441-6284.

`123`	an exact integer
`123.`	an approximate real number
`123.0000000000000`	an approximate real number with a certain precision
`123. + 0. I`	a complex number with approximate real number components

Several versions of the number 123.

You can distinguish different types of numbers in *Mathematica* by looking at their heads. (Although numbers in *Mathematica* have heads like other expressions, they do not have explicit elements which you can extract.)

The object 123 is taken to be an exact integer, with head `Integer`.

```
In[6]:= Head[123]
Out[6]= Integer
```

The presence of an explicit decimal point makes *Mathematica* treat `123.` as an approximate real number, with head `Real`.

```
In[7]:= Head[123.]
Out[7]= Real
```

`NumberQ[`*x*`]`	test whether *x* is any kind of number
`IntegerQ[`*x*`]`	test whether *x* is an integer
`EvenQ[`*x*`]`	test whether *x* is even
`OddQ[`*x*`]`	test whether *x* is odd
`PrimeQ[`*x*`]`	test whether *x* is a prime integer
`TrueQ[Head[`*x*`]==`*type*`]`	test the type of a number

Tests for different types of numbers.

`NumberQ[`*x*`]` tests for any kind of number.

```
In[8]:= NumberQ[5.6]
Out[8]= True
```

`5.` is treated as a `Real`, so `IntegerQ` gives `False`.

```
In[9]:= IntegerQ[5.]
Out[9]= False
```

If you use complex numbers extensively, there is one subtlety you should be aware

of. When you enter a number like `123.`, *Mathematica* treats it as an approximate real number, but assumes that its imaginary part is exactly zero. Sometimes you may want to enter approximate complex numbers with imaginary parts that are zero, but only to a certain precision.

When the imaginary part is the exact integer `0`, *Mathematica* simplifies complex numbers to real ones.

```
In[10]:= Head[ 123 + 0 I ]
Out[10]= Integer
```

Here the imaginary part is only zero to a certain precision, so *Mathematica* retains the complex number form.

```
In[11]:= Head[ 123. + 0. I ]
Out[11]= Complex
```

When the imaginary part of a complex number would print as `0.`, *Mathematica* just prints the real part.

```
In[12]:= 123. + 0. I
Out[12]= 123.
```

The distinction between complex numbers whose imaginary parts are exactly zero, or only zero to a certain precision, may seem like a pedantic one. However, when we discuss for example the interpretation of powers and roots of complex numbers in Section 3.2.7, the distinction will become significant.

One way to find out the type of a number in *Mathematica* is just to pick out its head using `Head[`*expr*`]`. For many purposes, however, it is better to use functions like `IntegerQ` which explicitly test for particular types. Functions like this are set up to return `True` if their argument is manifestly of the required type, and to return `False` otherwise. As a result, `IntegerQ[x]` will give `False`, unless you have assigned `x` an explicit integer value.

In doing symbolic computations, however, you may sometimes want to treat `x` as an integer, even though you have not assigned an explicit integer value to it. You can override the assumption that the symbol `x` is not an integer by explicitly making an assignment of the form `x/: IntegerQ[x] = True`. This assignment specifies that whenever you specifically test `x` with `IntegerQ`, it will give the result `True`. You should realize, however, that the assignment does not actually change the head of `x`, so that, for example, `x` will still not match `n_Integer`.

`x` does not explicitly have head `Integer`, so `IntegerQ` returns `False`.

```
In[13]:= IntegerQ[x]
Out[13]= False
```

This specifies that `x` is in fact an integer. The `x/:` specifies that the rule is associated with `x`, not `IntegerQ`.

```
In[14]:= x/: IntegerQ[x] = True
Out[14]= True
```

The definition overrides the default assumption that x is not an integer.

```
In[15]:= IntegerQ[x]
Out[15]= True
```